

Universal Recipe for Any Recursive Code

- The goal (what should the function return?) Define exactly what $f(n)$ represents
- The base case: what is the smallest, most trivial version of this problem that you can solve instantly? (e.g. If $n=0$, there is only 1 way).
- The Recursive relation: If you are at index i , how does the answer depend on $i-1$ or $i+1$?
- The choice!: At the current index, what are all the possible moves you can make?

Common pattern of Recursion

Multiple Recursive Call

- Logic: The current state depends on the sum or result of two or more previous state.
- Use cases: Fibonacci, Climbing Stairs.

The Pick/Not pick pattern (Subsets / Subsequence)

- Involving arrays or strings where you need to find combination.
- Logic: At every index, you have two choices, either include the element in your answer or leave it.
- Use case: Subsets, Power sets, combination sum.

The Exploration pattern

Used when navigating in 2D spaces.

- Logic: from your current (x, y) move in all valid direction (up, down, left and right)
- Use case: Rot in a Maze, Number of Island, word Search.

The divide and Conquer pattern

- Logic: Split the input into two half, solve them independently and merge the results.
- Use cases: Merge sort, binary tree traversal.

The backtracking pattern

- Logic: Make a choice (change state) $\rightarrow$ Recurse $\rightarrow$ undo the choice (backtrack state).
- Use cases: N-Queens, Sudoku Solver, Permutations.

1. The "Pick and Not Pick" Pattern

Goal: Used when you need to find all **Subsets, Subsequences, or Combinations**.

The Logic

Imagine you have a bag of items. For every single item, you have a binary choice:

1. **Pick:** Include the item in your current set and move to the next index.
2. **Not Pick:** Ignore the item and move to the next index.

By the time you reach the end of the array (the base case), you will have explored every possible combination because every element has been through these two choices.

The Interviewer's Secret: Time Complexity

- **Time Complexity:** $O(2^n)$. Why? Because for every element, you have 2 choices, and you do this n times.
- **Space Complexity:** $O(n)$, which is the depth of the recursion stack.

The "Universal" Java Template

This structure works for *Subsets*, *Combination Sum*, and *Target Sum*.

Java



```
public void findSubsequences(int index, int[] arr, List<Integer> currentList) {
    // 1. Base Case: If we've considered all elements
    if (index == arr.length) {
        System.out.println(new ArrayList<>(currentList));
        return;
    }

    // 2. The "Pick" Choice
    currentList.add(arr[index]); // Add the current element
    findSubsequences(index + 1, arr, currentList); // Move to next index

    // 3. The Backtracking Step
    // We must remove the element we just added before trying the "Not Pick" option
    currentList.remove(currentList.size() - 1);

    // 4. The "Not Pick" Choice
    findSubsequences(index + 1, arr, currentList); // Move to next index without
}
```

2. The "Exploration" Pattern (Grid/Matrix)

Goal: Used for **Pathfinding** or searching in a 2D space (Grid).

The Logic

Instead of just moving "next" in an array, you are at a coordinate `(row, col)`. Your "choices" are no longer pick/not pick; your choices are **directions**.

- Explore Up, Down, Left, and Right.
- **The Golden Rule:** You must mark a cell as "visited" so you don't get stuck in an infinite loop (going back and forth between two cells).

The Interviewer's Secret: Time Complexity

- **Time Complexity:** $O(4^{N \times M})$. In the worst case, from every cell, you try 4 directions.
- **Space Complexity:** $O(N \times M)$ for the recursion stack and visited array.

The "Universal" Java Template

This works for *Rat in a Maze*, *Number of Islands*, and *Word Search*.

The "Universal" Java Template

This works for *Rat in a Maze*, *Number of Islands*, and *Word Search*.

Java

```
public void solveGrid(int r, int c, int[][] grid, boolean[][] visited, String path) {
    // 1. Base Case: Out of bounds or blocked or already visited
    if (r < 0 || c < 0 || r >= grid.length || c >= grid[0].length ||
        grid[r][c] == 0 || visited[r][c]) {
        return;
    }

    // 2. Target Base Case: Reached the destination
    if (r == grid.length - 1 && c == grid[0].length - 1) {
        System.out.println("Path found: " + path);
        return;
    }

    // 3. Mark as Visited (Action)
    visited[r][c] = true;

    // 4. Explore all 4 possibilities
    solveGrid(r + 1, c, grid, visited, path + "D"); // Down
    solveGrid(r - 1, c, grid, visited, path + "U"); // Up
    solveGrid(r, c + 1, grid, visited, path + "R"); // Right
    solveGrid(r, c - 1, grid, visited, path + "L"); // Left

    // 5. Backtrack: Unmark visited so other paths can use this cell
    visited[r][c] = false;
}
```